

Deep Learning with TensorFlow

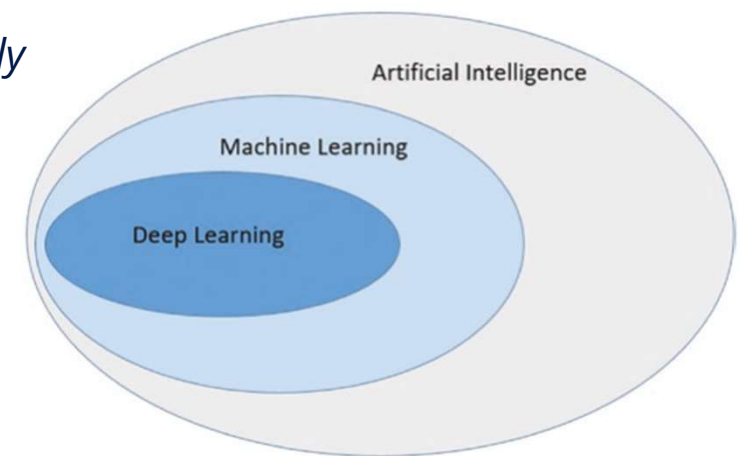
Faculty of Information Technology

Objectives

- ✓ Deep Learning
- ✓ Artificial Neural Networks
- ✓ TensorFlow
- ✓ Developing a Deep Learning Model with TensorFlow

AI – ML – DL

- Artificial Intelligence
 - *Automatic processing on a computer capable of performing operations that would seem to be exclusively relevant to human intelligence.*
- Machine Learning Is a Branch of Artificial Intelligence
 - *Machine learning (ML), with all its techniques and algorithms.*
 - able to learn (learning systems) to solve various problems that shortly before had been “considered exclusive to humans.”
- Deep Learning Is a Branch of Machine Learning
 - Deep learning systems are structured so as not to have these intrinsic characteristics in the model,
 - but these characteristics are extracted and detected by the system automatically as a result of learning itself.



Deep learning

- Deep learning is a computational modeling approach based on **multi-layer artificial neural networks** that learn hierarchical feature representations from data through nonlinear transformations and optimization using backpropagation.
- In data analytics, it enables:
 - Automated feature extraction
 - Pattern recognition in high-dimensional data
 - Predictive modeling at scale
 - End-to-end learning from raw data

How Deep Learning Fits into Data Analytics

- Data Analytics Hierarchy:
 - Data Collection
 - Data Cleaning
 - Exploratory Data Analysis (EDA)
 - Machine Learning
 - **Deep Learning (Advanced predictive modeling)**
- Deep learning is typically used when:
 - Data is large (Big Data scale)
 - Data is unstructured (images, text, audio)
 - Relationships are highly nonlinear

Common Deep Learning Architectures

- **Artificial Neural Networks (ANN)**
 - Used for structured/tabular predictive analytics.
- **Convolutional Neural Networks (CNN)**
 - Best for image analytics and computer vision.
- **Recurrent Neural Networks (RNN)**
 - Used for sequential/time-series data.
- **Transformers**
 - Used for natural language processing (e.g., large language models like OpenAI's models).

Applications in Data Analytics

- Customer churn prediction
- Fraud detection
- Demand forecasting
- Sentiment analysis
- Image classification
- Speech recognition
- Recommendation systems
- Example:
 - Netflix recommendations → deep learning models analyze user behavior patterns.
 - Banking fraud detection → detects abnormal transaction patterns.

- **Advantages**

- Handles complex nonlinear relationships
- Learns features automatically
- High performance with large datasets
- Works well with unstructured data

- **Limitations**

- Requires large amounts of data
- Computationally expensive (GPU/TPU needed)
- Less interpretable ("black box" problem)
- Risk of overfitting

Data Availability: Open Data Source, Internet of Things, and Big Data

- A major driver of deep learning's growth is the massive availability of data. Data is the core ingredient for neural networks, enabling both the training (learning phase) and testing (verification phase) processes.
- With the global expansion of the Internet, data is now widely accessible and continuously generated. In the past, only a limited number of organizations provided datasets. Today, technologies like the Internet of Things (IoT) allow sensors and smart devices to constantly collect and share data over networks. Additionally, social media platforms and search engines gather and analyze enormous volumes of real-time user data—commonly referred to as Big Data.
- As a result, large amounts of relevant data—often freely available as open data—are now accessible, making it easier to apply deep learning techniques to real-world problems.

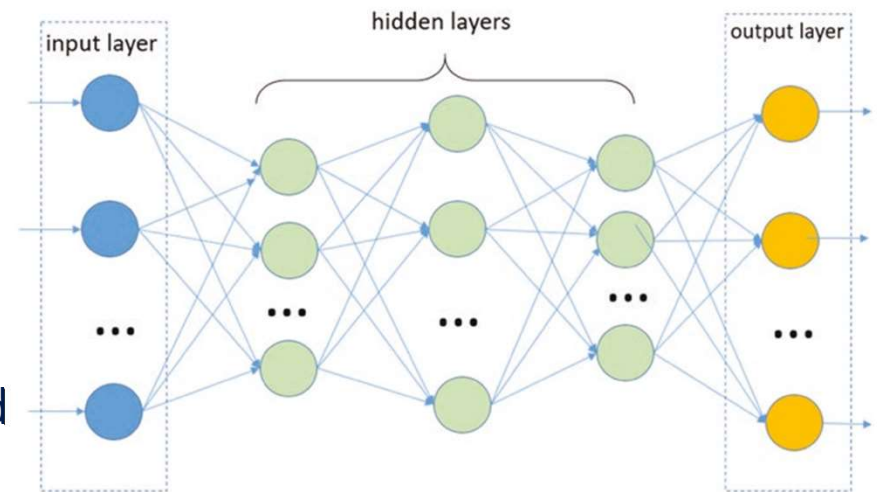
Deep Learning Python Frameworks

Several major frameworks support deep learning development:

- **TensorFlow** is an open-source numerical computing library based on data flow graphs, where nodes represent operations and edges represent tensors. It is highly flexible and supports distributed computing across CPUs and GPUs.
- **Caffe2** is a deep learning framework designed for simplicity and ease of use, enabling efficient model testing and GPU-powered cloud computation.
- **PyTorch** is a GPU-focused scientific framework known for its high efficiency and strong research capabilities. Although relatively newer, it has become a powerful tool for scientific research.
- **Theano** was a widely used Python library for defining and evaluating mathematical expressions and models. Although it is no longer actively developed, it remains influential due to its extensive historical use in research and applications.

Artificial Neural Networks

- **Artificial Neural Networks (ANNs)** are computational models inspired by the structure and functioning of the human brain. They are designed to recognize patterns, learn from data, and make predictions or decisions.
- An ANN is made up of interconnected processing units called **neurons**, organized in layers:
 - **Input Layer** – Receives raw data (features).
 - **Hidden Layer(s)** – Perform mathematical transformations and extract patterns.
 - **Output Layer** – Produces the final prediction or classification result.



How They Work

- Each connection between neurons has a **weight**, which represents the strength of the connection. During training:
 - The network makes predictions.
 - The error between predicted and actual results is calculated.
 - Weights are adjusted using optimization algorithms (like backpropagation and gradient descent).
 - Over time, the model learns patterns from data.

Key Characteristics

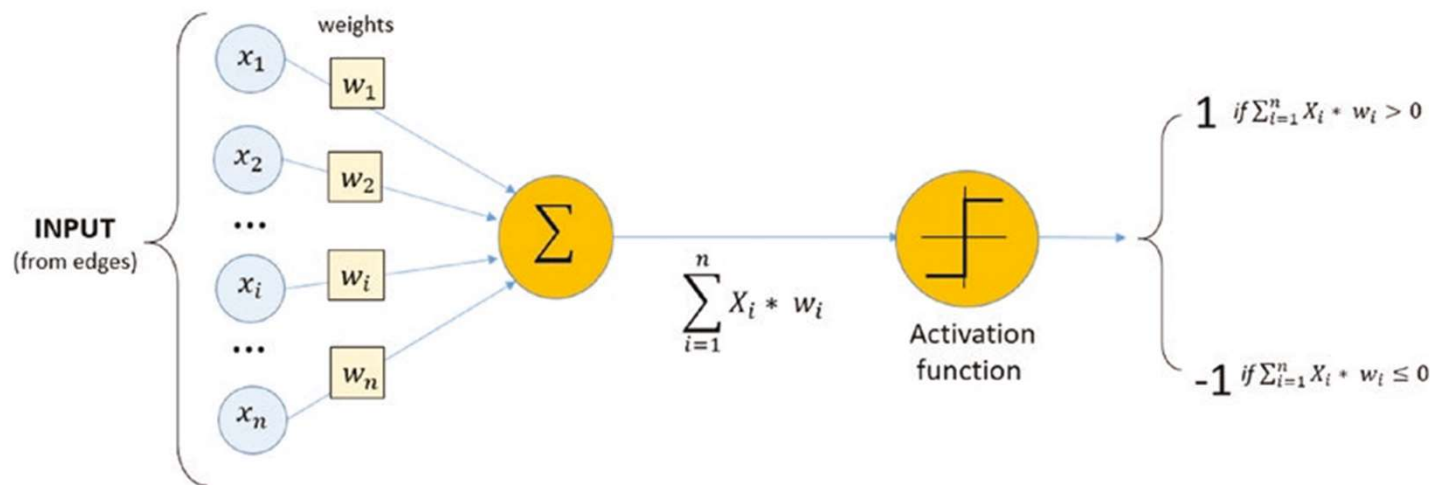
- **Nonlinear modeling** – Can learn complex relationships.
- **Data-driven learning** – Improves performance with more data.
- **Generalization ability** – Can make predictions on unseen data.

Applications

- ANNs are widely used in:
 - Image and speech recognition
 - Natural language processing
 - Medical diagnosis
 - Financial forecasting
 - Recommendation systems

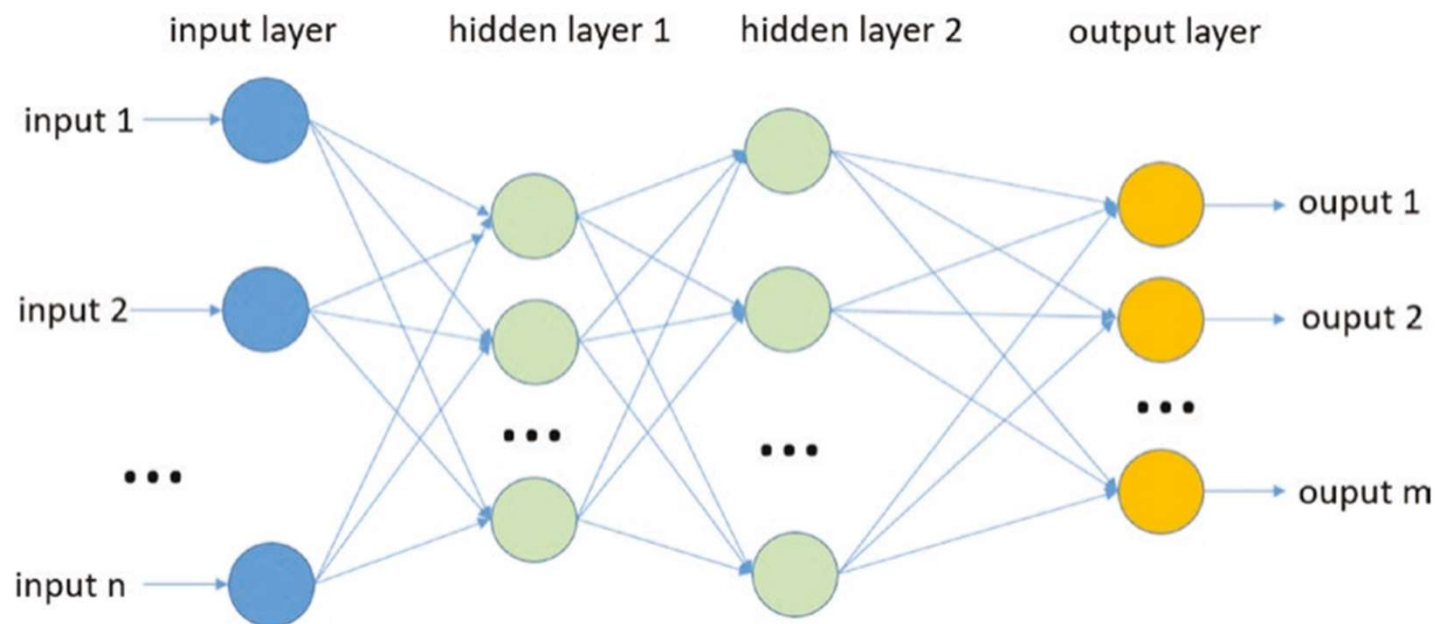
Single Layer Perceptron (SLP)

- The Single Layer Perceptron (SLP) structure is very simple; it is a two-layer neural network, without hidden layers, in which a number of input neurons send signals to an output neuron through different connections, each with its own weight.



Multilayer Perceptron (MLP)

- there are one or more hidden layers interposed between the input layer and the output layer.



TensorFlow

- **TensorFlow** is an open-source machine learning and deep learning framework developed by Google. In data analytics, it is widely used to build, train, and deploy deep learning models for extracting insights and making predictions from large datasets.
- TensorFlow enables:
 - **Numerical computation using tensors** (multi-dimensional arrays)
 - **Building neural networks** (ANNs, CNNs, RNNs, etc.)
 - **Training models efficiently on CPUs, GPUs, and TPUs**
 - **Deploying models** to web, cloud, and mobile applications
- It represents computations using **data flow graphs**, where:
 - Nodes = mathematical operations
 - Edges = tensors (data)

- In data analytics, TensorFlow helps to:
 - Perform **predictive analytics** (sales forecasting, churn prediction)
 - Apply **classification models** (fraud detection, sentiment analysis)
 - Analyze **images and videos**
 - Process **text and speech data**
 - Discover hidden patterns in Big Data
- It is especially powerful when working with **large-scale structured and unstructured data**.

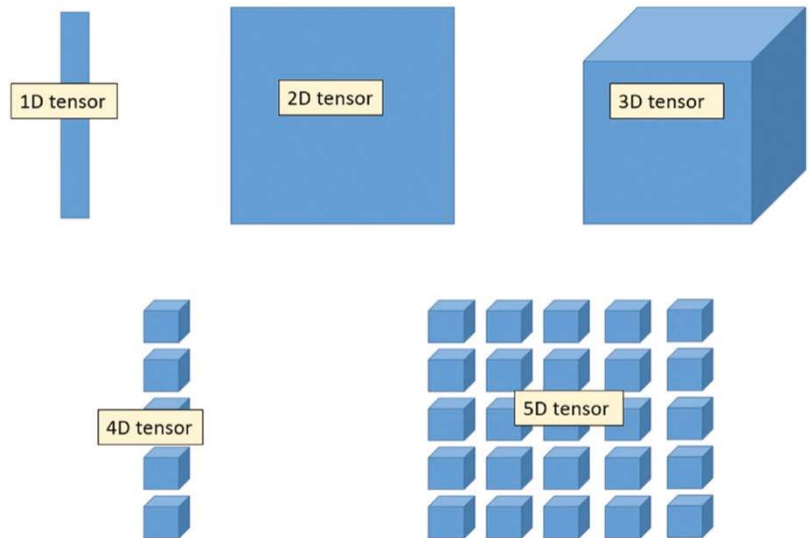
- **Why TensorFlow Is Important in Analytics**
 - Scalable to **big datasets**
 - Supports **distributed computing**
 - Integrates with Python ecosystem (NumPy, Pandas, etc.)
 - Production-ready for real-world business systems

Tensors

- The basic element of the TensorFlow library is the *tensor*.
- In fact, the data that follow the flow within the Data Flow Graph are tensors

- A tensor is identified by three parameters:

- **rank**—Dimension of the tensor
(a matrix has rank 2, a vector has rank 1)
- **shape**—Number of rows and columns
(e.g., (3,3) is a 3x3 matrix)
- **type**—The type of tensor elements



- Example

```
import numpy as np
t = np.arange(9).reshape((3,3))
t
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])
```

- Convert this multidimensional array into a TensorFlow tensor:
convert_to_tensor() function

```
import tensorflow as tf
```

```
tensor = tf.convert_to_tensor(t)
tensor
```

```
<tf.Tensor: shape=(3, 3), dtype=int64, numpy=
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])>
```

```
#convert the integer input into floating numbers
tensor2 = tf.convert_to_tensor(t, dtype='float64')
tensor2
```

```
<tf.Tensor: shape=(3, 3), dtype=float64, numpy=
array([[0., 1., 2.],
       [3., 4., 5.],
       [6., 7., 8.]])>
```

- initialize a tensor with all 0 values: the **tf.zeros()** method

```
t0 = tf.zeros((3,3), 'float64')
t0

<tf.Tensor: shape=(3, 3), dtype=float64, numpy=
array([[0., 0., 0.],
       [0., 0., 0.],
       [0., 0., 0.]])>
```

- a tensor with all values of 1: the **tf.ones()** method

```
t1 = tf.ones((3,3), 'float64')
t1

<tf.Tensor: shape=(3, 3), dtype=float64, numpy=
array([[1., 1., 1.],
       [1., 1., 1.],
       [1., 1., 1.]])>
```

- A 3x3 tensor with float values between 0 and 1

```
trand = tf.random.uniform((3, 3), minval=0, maxval=1, dtype=tf.float32)
trand

<tf.Tensor: shape=(3, 3), dtype=float32, numpy=
array([[0.6162684 , 0.878397  , 0.06351781],
       [0.988232  , 0.92503726, 0.3139465 ],
       [0.8863454 , 0.67332876, 0.64767253]], dtype=float32)>
```

- to create a tensor containing values that follow a **normal distribution** with a choice of **mean** and **standard deviation**: **tf.random_normal()** function.

```
tnorm = tf.random.normal((3, 3), mean=0, stddev=3)
tnorm
```

```
<tf.Tensor: shape=(3, 3), dtype=float32, numpy=
array([[ 5.429534 , -1.211594 , -2.0674512 ],
       [-0.84776545, -1.5808382 , -2.5333147 ],
       [-2.5902097 , -0.07615522,  1.5938687 ]], dtype=float32)>
```

Loading Data Into a Tensor from a pandas Dataframe

```
import pandas as pd
```

```
# define a simple dataframe
df = pd.DataFrame(np.array([[1, 2, 3],
[4, 5, 6],
[7, 8, 9]]),
columns=['a', 'b', 'c'])
df
```

	a	b	c
0	1	2	3
1	4	5	6
2	7	8	9

```
# tf.convert_to_tensor()
# accepts a pandas dataframe as an argument
tensor_df = tf.convert_to_tensor(df)
tensor_df
```

```
<tf.Tensor: shape=(3, 3), dtype=int64, numpy=
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])>
```

```
# the values of one or more columns.
tensor_df = tf.convert_to_tensor(df[['a', 'b']])
tensor_df
```

```
<tf.Tensor: shape=(3, 2), dtype=int64, numpy=
array([[1, 2],
       [4, 5],
       [7, 8]])>
```


Loading Data in a Tensor from a CSV File

```
from google.colab import files  
uploaded = files.upload()
```

training_data.csv
training_data.csv(text/csv) - 103 bytes, last modified: 3/2/2026 - 100% done
Saving training_data.csv to training_data (3).csv

```
df = pd.read_csv('training_data (3).csv')  
df
```

	X	Y	label
0	1.0	3.0	0
1	1.0	2.0	0
2	1.0	1.5	0
3	1.5	2.0	0
4	2.0	3.0	0
5	2.5	1.5	1
6	2.0	1.0	1
7	3.0	1.0	1
8	3.0	2.0	1
9	3.5	1.0	1
10	3.5	3.0	1

*The pandas dataframe
containing the training
dataset*

- **Training datasets** are composed of two parts—the features and the labels: extract two distinct tensors from the dataframe, one for the **features** and one for the **labels**.

```
df_features = df.copy()
df_labels = df_features.pop('label')
data_features = tf.convert_to_tensor(df_features)
data_features
```

```
<tf.Tensor: shape=(11, 2), dtype=float64, numpy=
array([[1. , 3. ],
       [1. , 2. ],
       [1. , 1.5],
       [1.5, 2. ],
       [2. , 3. ],
       [2.5, 1.5],
       [2. , 1. ],
       [3. , 1. ],
       [3. , 2. ],
       [3.5, 1. ],
       [3.5, 3. ]])>
```

```
#the tensor of the labels
data_labels = tf.convert_to_tensor(df_labels)
data_labels
```

```
<tf.Tensor: shape=(11,), dtype=int64, numpy=array([0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1])>
```

Operation on Tensors

- Most mathematical calculations on tensors are based on the sum and multiplication between tensors.
 - Sum 2 tensors: `tf.add()` function
 - Multiplication 2 tensors: `tf.matmul()` function

```
t1 = tf.random.uniform((3, 3), minval=0, maxval=1, dtype=tf.float32)
t1
```

```
<tf.Tensor: shape=(3, 3), dtype=float32, numpy=
array([[0.66229534, 0.08925521, 0.95199454],
       [0.42476964, 0.2680359 , 0.10027885],
       [0.14164472, 0.6910136 , 0.48101664]], dtype=float32)>
```

```
t2 = tf.random.uniform((3, 3), minval=0, maxval=1, dtype=tf.float32)
t2
```

```
<tf.Tensor: shape=(3, 3), dtype=float32, numpy=
array([[0.56602013, 0.07339847, 0.43035054],
       [0.6826912 , 0.11139405, 0.62417746],
       [0.9985645 , 0.6278074 , 0.567377  ]], dtype=float32)>
```

```
sum = tf.add(t1,t2)
sum
```

```
<tf.Tensor: shape=(3, 3), dtype=float32, numpy=
array([[1.2283155 , 0.16265368, 1.3823451 ],
       [1.1074609 , 0.37942994, 0.7244563 ],
       [1.1402092 , 1.318821 , 1.0483936 ]], dtype=float32)>
```

```
mul = tf.matmul(t1,t2)
mul
```

```
<tf.Tensor: shape=(3, 3), dtype=float32, numpy=
array([[1.3864342 , 0.6562232 , 0.88087004],
       [0.52354884, 0.12399085, 0.4069977 ],
       [1.0322489 , 0.3893571 , 0.7651897 ]], dtype=float32)>
```

Developing a Deep Learning Model with TensorFlow

- After defining tensors (the core data structures in TensorFlow), building a deep learning model in **TensorFlow 2.x** follows these key steps:
 - **Define Tensors** – Prepare and organize the training and testing datasets.
 - **Build the Model** – Design the neural network architecture (layers, neurons, activation functions).
 - **Compile the Model** – Configure the learning process by selecting the optimizer, loss function, and evaluation metrics.
 - **Train the Model** – Fit the model to the training data so it can learn patterns.
 - **Test the Model** – Evaluate performance using the testing dataset.
 - **Make Predictions** – Use the trained model to generate predictions on new data.
- In short, TensorFlow's workflow moves from data preparation to model design, learning, evaluation, and finally real-world prediction.

Single Layer Perceptron with TensorFlow

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
```

- The set of data that you will study is a set of 11 points distributed in a Cartesian axis, divided into two classes of membership. The first six belong to the first class, the other five to the second. The coordinates (x, y) of the points are contained within a NumPy inputX array, while the class to which they belong is indicated in inputY. This is a list of two-element arrays, with an element for each class they belong to. The value 1 in the first or second element indicates the class to which it belongs.
- If the element has value [1.0], it will belong to the first class. If it has value [0,1], it belongs to the second class. The fact that they are floating point values is due to the optimization calculation of deep learning. You will see later that the test results of the neural networks are floating numbers, indicating the probability that an element belongs to the first or second class.

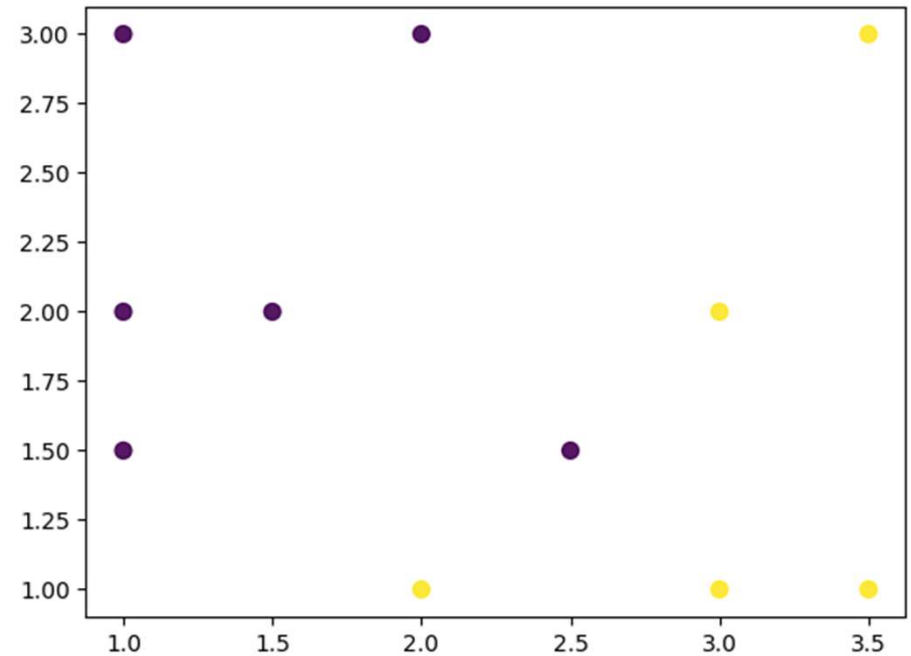
```
#Training set
inputX = np.array([[1.,3.],[1.,2.],[1.,1.5],[1.5,2.],[2.,3.],[2.5,1.5],
[2.,1.],[3.,1.],[3.,2.],[3.5,1.],[3.5,3.]])
inputY = [[1.,0.]]*6+ [[0.,1.]]*5
print(inputX)
print(inputY)
```

```
[[1.  3. ]
 [1.  2. ]
 [1.  1.5]
 [1.5 2. ]
 [2.  3. ]
 [2.5 1.5]
 [2.  1. ]
 [3.  1. ]
 [3.  2. ]
 [3.5 1. ]
 [3.5 3. ]]
[[1.0, 0.0], [1.0, 0.0], [1.0, 0.0], [1.0, 0.0], [1.0, 0.0], [1.0, 0.0],
```

The training set is a set of Cartesian points divided into two classes of membership (light and dark)

```
yc = [0]*6 + [1]*5
print(yc)
plt.scatter(inputX[:,0],inputX[:,1],c=yc, s=50, alpha=0.9)
plt.show()
```

```
[0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1]
```



- convert the values of the training arrays into tensor

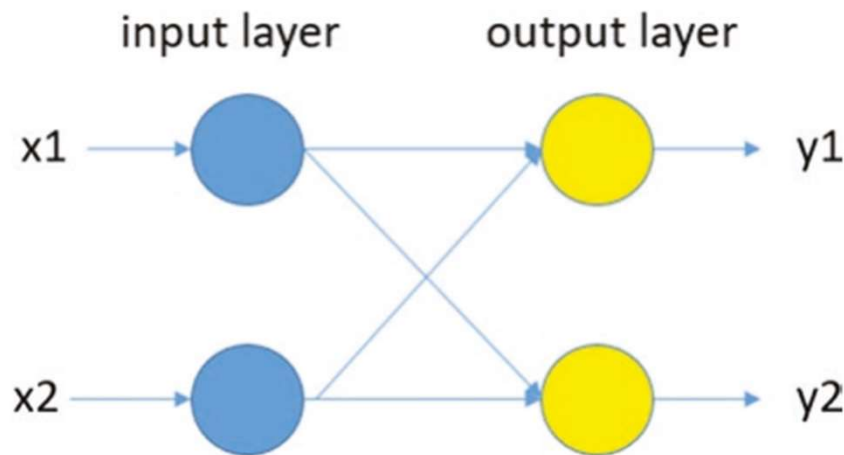
```
train_features = tf.convert_to_tensor(inputX)
train_labels = tf.convert_to_tensor(inputY)
train_features

<tf.Tensor: shape=(11, 2), dtype=float64, numpy=
array([[1. , 3. ],
       [1. , 2. ],
       [1. , 1.5],
       [1.5, 2. ],
       [2. , 3. ],
       [2.5, 1.5],
       [2. , 1. ],
       [3. , 1. ],
       [3. , 2. ],
       [3.5, 1. ],
       [3.5, 3. ]])>
```

```
train_labels

<tf.Tensor: shape=(11, 2), dtype=float32, numpy=
array([[1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.]], dtype=float32)>
```

- Build the Single Layer Perceptron model based on the neural network, adding the various Keras layers to the model definition.



```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(2),
    tf.keras.layers.Dense(2)
])
```

```
model.compile(optimizer='SGD',
              loss=tf.keras.losses.BinaryCrossentropy(from_logits=True),
              metrics=['accuracy'])
```

```
# choose BinaryCrossentropy as
# the function loss for the compilation.
model.compile(optimizer='SGD',
              loss=tf.keras.losses.BinaryCrossentropy(from_logits=True),
              metrics=['accuracy'])
```

```
#Once the model is compiled, you can move on to training it
#epochs=200 - fast
#longtime ...
h = model.fit(train_features, train_labels, epochs=2000)
```

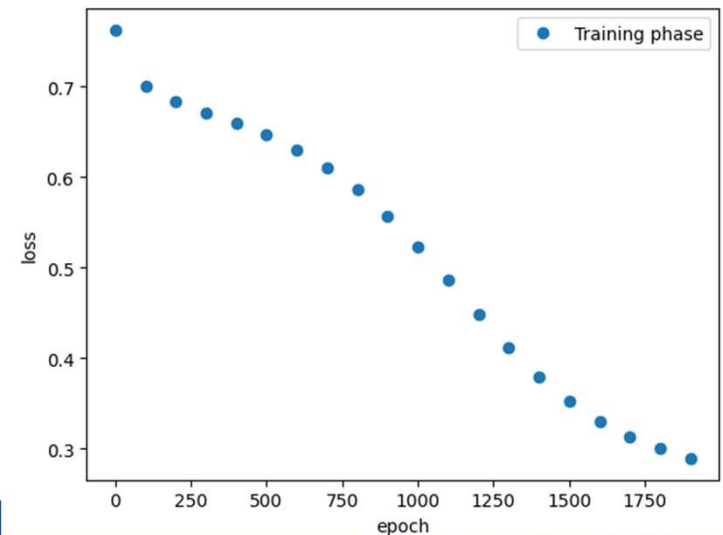
```
Epoch 1972/2000
1/1 _____ 0s 137ms/step - accuracy: 0.9091 - loss: 0.2830
Epoch 1973/2000
1/1 _____ 0s 141ms/step - accuracy: 0.9091 - loss: 0.2829
Epoch 1974/2000
1/1 _____ 0s 262ms/step - accuracy: 0.9091 - loss: 0.2828
Epoch 1975/2000
1/1 _____ 0s 152ms/step - accuracy: 0.9091 - loss: 0.2827
```


- Extract from the history variable the loss values corresponding to the various periods.

```
acc_set = h.history['loss']  
epoch_set = h.epoch
```

- Arrange these values in a plotting chart to see the learning progress graphically: the matplotlib library.

```
# return list of every 100th item in a larger list  
plt.plot(epoch_set[0::100], acc_set[0::100], 'o', label='Training phase')  
plt.ylabel('loss')  
plt.xlabel('epoch')  
plt.legend()
```



- Now move on to the testing phase, defining the other two tensors.

```
testX = np.array([[1.,2.25],[1.25,3.],  
[2,2.5],[2.25,2.75],  
[2.5,3.],[2.,0.9],  
[2.5,1.2],[3.,1.25],  
[3.,1.5],[3.5,2.],  
[3.5,2.5]])  
testY = [[1.,0.]]*5 + [[0.,1.]]*6  
test_features = tf.convert_to_tensor(testX)  
test_labels = tf.convert_to_tensor(testY)
```

- Evaluate the accuracy of the newly educated SLP model

```
test_loss, test_acc = model.evaluate(test_features, test_labels, verbose=2)  
  
1/1 - 0s - 372ms/step - accuracy: 1.0000 - loss: 0.1698
```

```
#To this end, the program simulates experimental data,  
#creating points on the Cartesian plane that are completely random.  
#For example, you can generate an array containing 1,000 random points.  
exp_features = 3*np.random.random((1000,2))
```

```
#Now you extend the model with Softmax to obtain  
#an output of the probability of the different points  
#belonging to the two classes.  
probability_model = tf.keras.Sequential([  
    model,  
    tf.keras.layers.Softmax()  
)
```

```
#make the predictions of the experimental data with the model you just extended  
predictions = probability_model.predict(exp_features)
```

32/32 ————— 0s 4ms/step

```
#determine the probability of a single point belonging to the two classes.  
predictions[0]
```

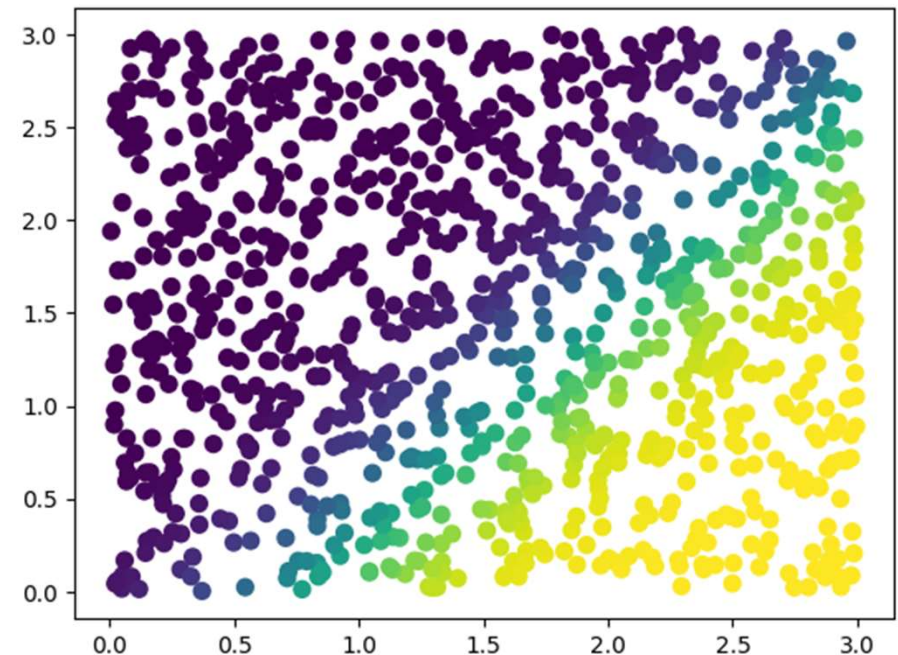
```
array([0.78194934, 0.21805064], dtype=float32)
```

```
#If, on the other hand, want to know directly  
#which of the two classes it belongs to, we write the  
#following code obtaining the class  
#it belongs to (0 for the first and 1 for the second)  
np.argmax(predictions[0])
```

```
np.int64(0)
```

- Instead of analyzing point by point in a textual way, there is a graphical way to visualize the result of these predictions.

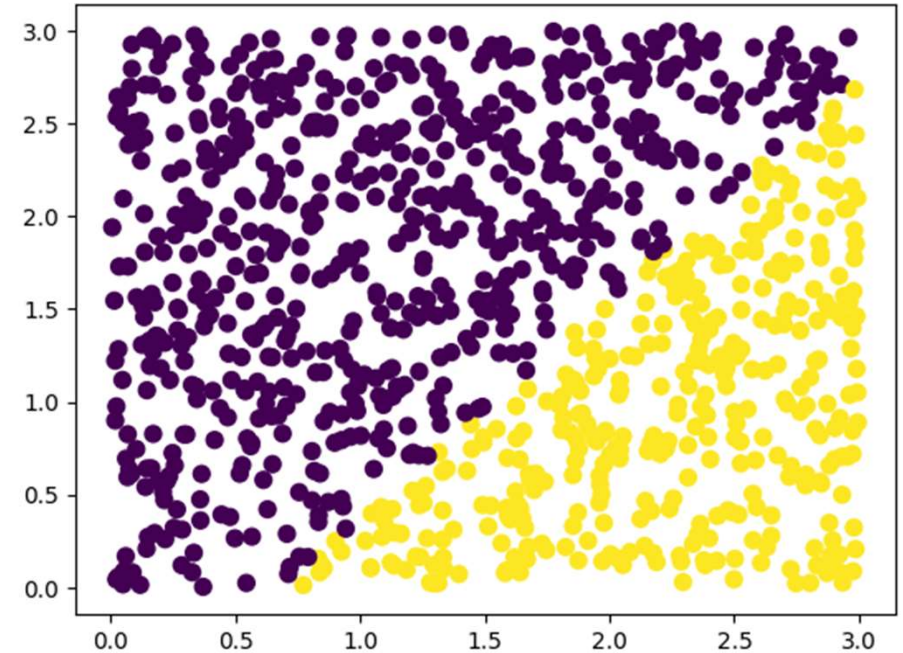
```
#Instead of analyzing point by point in a textual way,  
#there is a graphical way to visualize the result of these predictions.  
yc = predictions[:,1]  
plt.scatter(exp_features[:,0],exp_features[:,1],c=yc, s=50, alpha=1)  
plt.show()
```



A scatterplot with all the experimental points and the estimate of the classes to which they belong

- modify the previous scatterplot by requiring that each point belong to one or another class, according to the most probable option.

```
#modify the previous scatterplot by requiring that  
#each point belong to one or another class,  
#according to the most probable option.  
yc = np.round(predictions[:,1])  
plt.scatter(exp_features[:,0],exp_features[:,1],c=yc, s=50, alpha=1)  
plt.show()
```



*The points delimit the two regions
corresponding to the two classes*

Multilayer Perceptron (with One Hidden Layer) with TensorFlow

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(2),
    tf.keras.layers.Dense(2),
    tf.keras.layers.Dense(2)
])
```

```
#For MLP neural networks, a good choice is the Adam optimization method.
model.compile(optimizer='Adam',
    loss=tf.keras.losses.BinaryCrossentropy(from_logits=True),
    metrics=['accuracy'])
```

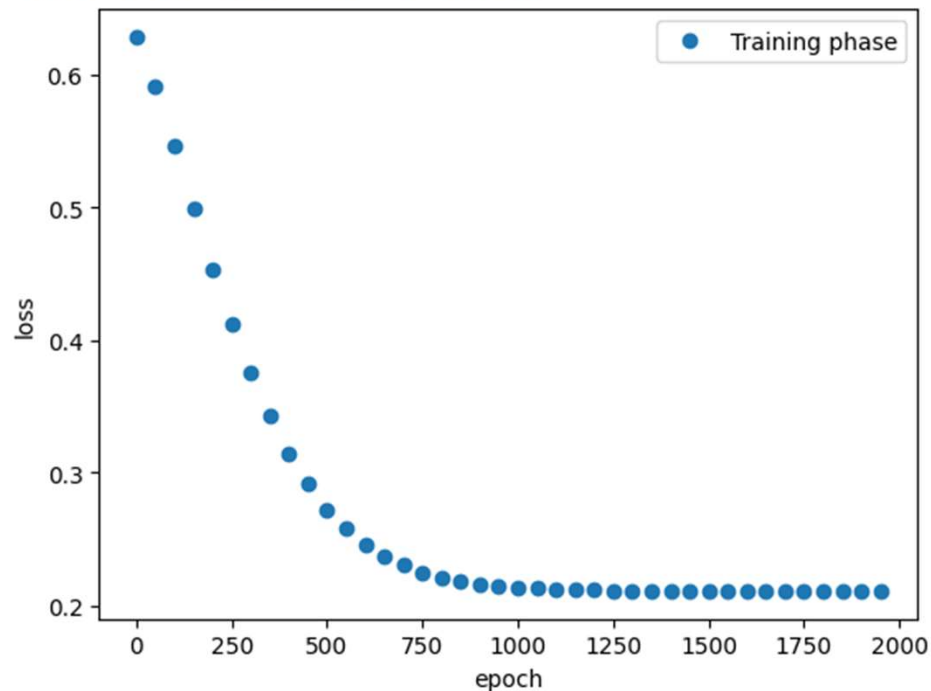
```
# train it using the same training dataset. [Fast: epochs=200]
h = model.fit(train_features, train_labels, epochs=2000)
```

```
1/1 _____ 0s 74ms/step - accuracy: 0.9091 - loss: 0.2110
Epoch 1807/2000
1/1 _____ 0s 74ms/step - accuracy: 0.9091 - loss: 0.2110
Epoch 1808/2000
1/1 _____ 0s 137ms/step - accuracy: 0.9091 - loss: 0.2110
Epoch 1809/2000
1/1 _____ 0s 139ms/step - accuracy: 0.9091 - loss: 0.2110
Epoch 1810/2000
1/1 _____ 0s 139ms/step - accuracy: 0.9091 - loss: 0.2110
Epoch 1811/2000
1/1 _____ 0s 140ms/step - accuracy: 0.9091 - loss: 0.2110
Epoch 1812/2000
1/1 _____ 0s 149ms/step - accuracy: 0.9091 - loss: 0.2110
Epoch 1813/2000
1/1 _____ 0s 74ms/step - accuracy: 0.9091 - loss: 0.2110
Epoch 1814/2000
1/1 _____ 0s 141ms/step - accuracy: 0.9091 - loss: 0.2110
```



```
#Extract the training history and
#create the same graph showing the behavior of the model
#during the training process.
acc_set = h.history['loss']
epoch_set = h.epoch# return list of every 50th item in a larger list
plt.plot(epoch_set[0::50],acc_set[0::50], 'o', label='Training phase')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend()
```

<matplotlib.legend.Legend at 0x7e849e9ee450>



▶ #As the result of testing this last model:
 #get an accuracy of 100 percent and a loss value of 01
 test_loss, test_acc = model.evaluate(test_features, test_labels, verbose=2)

... 1/1 - 0s - 197ms/step - accuracy: 1.0000 - loss: 0.0954

Multilayer Perceptron (with Two Hidden Layers) with TensorFlow

- adding two neurons to the first hidden layer (four in all) and adding a second hidden layer with two neurons.

```
model = tf.keras.Sequential([  
    tf.keras.layers.Dense(2),  
    tf.keras.layers.Dense(4),  
    tf.keras.layers.Dense(2),  
    tf.keras.layers.Dense(2)  
])
```

```
model.compile(optimizer='Adam',  
              loss=tf.keras.losses.BinaryCrossentropy(from_logits=True),  
              metrics=['accuracy'])
```

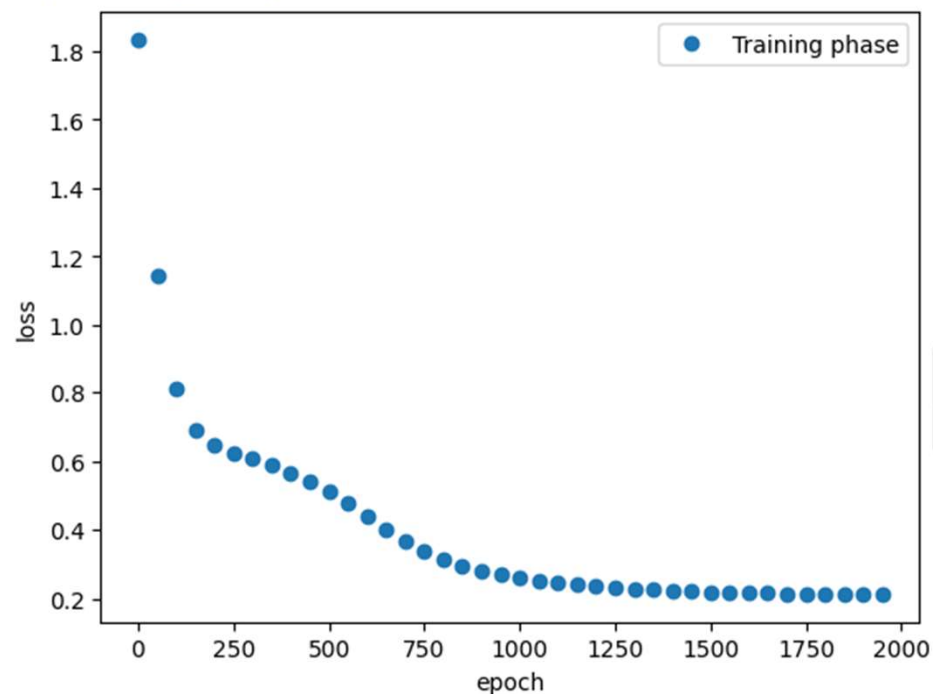
```
#train the new two hidden layer MLP model with the same training dataset  
h = model.fit(train_features, train_labels, epochs=2000)
```

```
Epoch 1972/2000  
1/1 _____ 0s 116ms/step - accuracy: 0.9091 - loss: 0.2125  
Epoch 1973/2000  
1/1 _____ 0s 95ms/step - accuracy: 0.9091 - loss: 0.2125  
Epoch 1974/2000  
1/1 _____ 0s 107ms/step - accuracy: 0.9091 - loss: 0.2125  
Epoch 1975/2000  
1/1 _____ 0s 117ms/step - accuracy: 0.9091 - loss: 0.2125  
Epoch 1976/2000  
1/1 _____ 0s 165ms/step - accuracy: 0.9091 - loss: 0.2125  
Epoch 1977/2000  
1/1 _____ 0s 132ms/step - accuracy: 0.9091 - loss: 0.2125  
Epoch 1978/2000  
1/1 _____ 0s 116ms/step - accuracy: 0.9091 - loss: 0.2125
```



```
#analyze the learning phase of the neural network by
#displaying the loss values as the epochs increase in a chart.
acc_set = h.history['loss']
epoch_set = h.epoch
# return list of every 50th item in a larger list
plt.plot(epoch_set[0::50],acc_set[0::50], 'o', label='Training phase')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend()
```

<matplotlib.legend.Legend at 0x7e849f3e7230>



```
test_loss, test_acc = model.evaluate(test_features, test_labels, verbose=2)
```

```
1/1 - 0s - 342ms/step - accuracy: 1.0000 - loss: 0.0984
```

